University of Science - VNUHCM

Faculty of Information Technology

Software Testing

Software Testing Project Report

HW3 - Data Generation

Authors:
Võ Lê Việt Tú (22127435)
vlvtu22@clc.fitus.edu.vn

Supervisors:
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

June 24, 2025

Table of Contents

1 Group Information

Group ID: 07

Member Name	Student ID	Assigned Tables	Status
Cao Uyển Nhi	22127310	- Invoices	Done
		- Invoice Items	Done
Lưu Thanh Thuý	22127410	- Brands	Done
		- Categories	Done
Nguyễn Phước Minh Trí	22127424	- Product Image	Done
		- Personal Access Token	Done
Võ Lê Việt Tú	22127435	- Product	Done
		- Favorites	Done
Trần Thị Cát Tường	22127444	- User	Done
		- Contact Requests	Done

2 Introduction

This report describes the process of generating synthetic data for an e-commerce tools database. Instead of using a third-party data generation tool, I developed a custom-built solution using Python. The primary libraries utilized were Faker and Pandas to create realistic, meaningful data for products, brands, categories, and user favorites.

3 Custom Data Generation Tool

The data generation was accomplished using custom Python scripts that leverage the Faker library to create realistic data along with domain-specific logic to ensure the data is meaningful and consistent.

3.1 Implementation Overview

The custom tool consists of separate Python scripts for generating different data entities:

1. `products.py` - Generates product data
2. `favorites.py` - Generates user favorite product relationships
3. Additional scripts for brands and categories (not shown in this report)

4 Data Fields and Randomization Rules

4.1 Products Dataset (600 rows)

Field	Data Type	Randomization Rules
id	Integer	Sequential IDs from 1 to 600
name	String	Combination of adjective + tool name + model number
description	Text	Generated using templates with random features and benefits
stock	Integer	Random value between 0 and 500
price	Decimal	Random value between 10.0 and 1000.0, rounded to 2 decimal places
is_location_offer	Boolean (0/1)	Random value of 0 or 1
is_rental	Boolean (0/1)	Random value of 0 or 1
brand_id	Integer	References a valid brand ID
category_id	Integer	References a valid category ID
product_image_id	Integer	Random value between 1 and 500
created_at	DateTime	Random date within the last year
updated_at	DateTime	Created date plus 0-30 days

4.2 Favorites Dataset (600 rows)

Field	Data Type	Randomization Rules
id	Integer	Sequential IDs from 1 to 600
user_id	Integer	Random value between 1 and 500
product_id	Integer	Random value between 1 and 600
created_at	DateTime	Random date within the last 2 years
updated_at	DateTime	Created date plus 0-300 days

5 Code Explanation

5.1 Products Generation

The product generation code is designed around a modular architecture that creates realistic and contextually appropriate tool products. Key aspects of the product generation:

1. **Data Sources and Libraries**:
    - Pandas for data manipulation and CSV operations
    - Faker for generating realistic random data
    - Random module for basic randomization
    - Datetime for timestamp handling
2. **Reference Data Structure**:
    - **Product Adjectives**: A curated list of 24 descriptive adjectives specific to tools and hardware
    - **Product Features**: 18 detailed features focusing on ergonomics, durability, efficiency, and usability
    - **Category-Specific Names**: A dictionary mapping 5 major tool categories to appropriate tool types
    - **Generic Tool Names**: Fallback options for categories without specific mappings
3. **Name Generation Logic**:
    - Products are named using a three-part structure:
        - **Adjective**: Selected from the product_adjectives list (e.g., "Professional", "Heavy-Duty")
        - **Tool Type**: Selected based on the product's category (e.g., "Drill" for Power Tools)
        - **Model Number**: A randomly generated alphanumeric code (e.g., "A123")
    - The system intelligently selects tool names that match the product category
4. **Description Generation System**:
    - Descriptions follow a consistent three-part template:

- **Introduction**: References the product type and brand name
- **Features Section**: Incorporates 2-3 randomly selected features with proper grammatical formatting
- **Use Case**: Concludes with an appropriate application context for the tool

5. **Data Integrity Measures**:

   - All products reference valid brand and category IDs from the source data
   - Price values are realistic for tools (10.0 to 1000.0) and properly rounded
   - Created and updated timestamps maintain logical chronology
   - Each product receives a unique sequential ID

6. **Timestamp Generation**:

   - Created dates are distributed throughout the past year
   - Updated dates occur 0-30 days after creation, maintaining temporal consistency

7. **Output Formatting**:

   - All data is structured into a pandas DataFrame
   - Final output is encoded in UTF-8 and saved as CSV

5.2 Favorites Generation

The favorites generation script creates relationships between users and products, ensuring that each user-product favorite relationship is unique. This is a critical aspect of maintaining data integrity in the e-commerce database. Key aspects of the favorites generation:

1. **Data Uniqueness System**:

   - The script uses a Python set (`generated_pairs`) to track which user-product combinations have already been generated
   - This ensures that no user favorites the same product twice, which would violate data integrity
   - The set lookup operation is O(1), making it efficient even with large datasets

2. **Controlled Randomization**:

   - User IDs are randomly selected from a range of 1-500
   - Product IDs are randomly selected from a range of 1-600
   - The script will continue generating random combinations until it reaches the target of 600 unique favorites

3. **Intelligent Loop Design**:

   - The while loop continues until exactly 600 unique favorites are generated
   - The continue statement skips iterations where a duplicate would be created
   - This approach ensures we get exactly the desired number of records

4. **Realistic Temporal Data**:
    - Created timestamps span the last 2 years, providing a realistic history of user activity
    - Updated timestamps occur 0-300 days after creation, representing realistic user interaction patterns
    - All timestamps are properly formatted for database compatibility
5. **Data Structure and Output**:
    - Records are stored in a list of dictionaries for flexibility
    - Pandas is used to convert the data to a DataFrame and handle CSV formatting
    - Output includes confirmation of the exact number of records created

6 Process and Steps

The data generation process followed a systematic approach designed to produce realistic, coherent datasets suitable for an e-commerce platform specializing in tools and hardware. Below is a detailed breakdown of each phase:

6.1 Requirements Analysis and Planning

- **Initial Requirements Assessment:**
    - Identified the need for at least 500 rows of meaningful data
    - Determined that the dataset should represent an e-commerce tools platform
    - Decided to focus on products and user favorites as primary entities
    - Established that data must be realistic and domain-appropriate
- **Data Schema Planning:**
    - Created entity relationship diagrams to visualize the database structure
    - Defined primary and foreign key relationships between tables
    - Decided on specific data types and constraints for each field
    - Identified which fields would require special handling (like unique constraints)
- **Technology Selection:**
    - Evaluated several data generation tools and approaches:
        * Considered commercial tools like Mockaroo and Faker.js
        * Assessed open-source libraries like Python Faker
        * Explored AI-assisted data generation options
    - Selected Python with Faker and Pandas based on:
        * Flexibility for custom logic implementation
        * Robust support for various data types
        * Ability to enforce relationships between datasets
        * Strong CSV output capabilities
        * Familiarity with the ecosystem

6.2 Reference Data Preparation

- **Brand and Category Datasets:**
  - Created a brands.csv file with realistic tool manufacturers:
    - Included well-known brands like DeWalt, Milwaukee, Bosch, and Makita
    - Added variety with both premium and budget-oriented brands
    - Assigned sequential IDs for reference integrity
  - Created a categories.csv file with a hierarchical category structure:
    - Divided into major categories (Power Tools, Hand Tools, etc.)
    - Added subcategories where appropriate
    - Assigned sequential IDs for reference integrity
- **Product Naming Components:**
  - Researched and compiled domain-specific terminology:
    - Collected tool-appropriate adjectives from industry catalogs
    - Created category-specific naming conventions
    - Developed a model number generation system
- **Product Description Elements:**
  - Analyzed real product descriptions from major tool retailers
  - Identified common structural patterns in tool descriptions
  - Created lists of realistic features and benefits
  - Developed templates that would produce coherent, varied descriptions

6.3 Script Development and Implementation

- **Core Framework Development:**
  - Set up the project structure with separate scripts for each entity
  - Established common conventions for random seed management
  - Created utility functions for timestamp generation and formatting
- **Products Generation Implementation:**
  - Developed the category-specific name generation system
  - Implemented the three-part product description templates
  - Created logic for realistic price and stock level generation
  - Added proper referential integrity with brands and categories
- **Favorites Generation Implementation:**
  - Created the uniqueness enforcement system using sets
  - Implemented realistic timestamp generation with proper sequencing
  - Ensured references to valid user and product IDs
  - Added sequential ID assignment logic

- **Data Validation Logic:**
  - Added checks to ensure no duplicate user-product pairs in favorites
  - Implemented validation for referential integrity
  - Added range constraints for numeric values

6.4 Testing and Refinement

- **Initial Test Runs:**
  - Generated small test datasets (50 records) to evaluate output quality
  - Manually reviewed sample entries for realism and coherence
  - Checked for any patterns or biases in the generated data
- **Script Refinement:**
  - Adjusted randomization ranges based on initial tests
  - Enhanced description templates for more variety
  - Fine-tuned timestamp distribution for more realistic patterns
  - Optimized loop structures for better performance with larger datasets
- **Final Validation:**
  - Generated the full datasets (600 records each)
  - Performed automated validation checks:
    - Uniqueness constraints
    - Referential integrity
    - Data type consistency
    - Range validations
  - Conducted manual spot-checks of random samples

6.5 Data Generation and Export

- **Final Production Run:**
  - Executed the scripts in proper sequence:
    1. Generated reference data (brands, categories)
    2. Generated products data referencing brands and categories
    3. Generated favorites data referencing products
  - Monitored execution for any errors or performance issues
- **Data Export and Formatting:**
  - Exported all datasets to CSV format with consistent encoding (UTF-8)
  - Validated CSV files for proper formatting and completeness
  - Performed spot checks on the final output files
- **Documentation:**

- Captured screenshots of the generation process
- Documented the randomization rules and ranges
- Prepared sample data excerpts for the report
- Compiled implementation details and methodology notes

The methodical approach ensured that the generated data not only met the quantity requirements (500+ rows) but also maintained high quality in terms of realism, coherence, and referential integrity across the various entities in the database.

7 Sample Data

7.1 Sample Products

id	name	description	stock	price	brand_id	category_id
1	Premium Drill F421	This premium drill from DeWalt is designed for professional and DIY enthusiasts alike. Features include ergonomic handle for reduced fatigue and precision-engineered components for superior performance. Perfect for construction sites.	127	199.99	3	5
2	Heavy-Duty Socket Set Z789	This heavy-duty socket set from Milwaukee is designed for professional and DIY enthusiasts alike. Features include rust-resistant coating for longer tool life and quick-release mechanism for easy operation. Perfect for automotive applications.	84	149.95	7	12

id	name	description	stock	price	brand_id	category_id
3	Industrial Heat Gun C345	This industrial heat gun from Bosch is designed for professional and DIY enthusiasts alike. Features include variable speed control for versatility and anti-slip grip for safer operation. Perfect for industrial applications.	42	79.99	5	4

7.2 Sample Favorites

id	user_id	product_id	created_at	updated_at
1	143	257	2023-07-15 14:22:17	2024-02-22 08:45:33
2	85	412	2023-09-04 11:08:42	2024-05-19 17:30:11
3	219	78	2024-01-22 09:17:36	2024-04-11 14:52:08

8 Self-Evaluation (Data Generation)

Criteria	Self-Evaluation	Notes
2 Tables Selection	1.0 / 1.0	Selected two important tables: `products` and `favorites`.
Sample Data	2.0 / 2.0	All data is meaningful, realistic, and aligned with UI/business context.
Data Generation Report	1.0 / 1.0	Report includes field rules, tools used, prompts, process, and samples.

9 Execution Screenshots

```python
        product = {
            "id": i,
            "name": name,
            "description": description,
            "stock": random.randint(0, 500),
            "price": round(random.uniform(10.0, 1000.0), 2),
            "is_location_offer": random.randint(0, 1),
            "is_rental": random.randint(0, 1),
            "brand_id": brand_id,
            "category_id": category_id,
            "product_image_id": random.randint(1, 500),
            "created_at": created_at.strftime('%Y-%m-%d %H:%M:%S'),
            "updated_at": updated_at.strftime('%Y-%m-%d %H:%M:%S')
        }
        products.append(product)
    return products

product_data = generate_products(600)

df = pd.DataFrame(product_data)
df.to_csv("products.csv", index=False)
print("Done! Data saved to products.csv")
```

Figure 1: The products.py script

```python
    favorites.append({
        "id": len(favorites) + 1,
        "user_id": user_id,
        "product_id": product_id,
        "created_at": created_at.strftime('%Y-%m-%d %H:%M:%S'),
        "updated_at": updated_at.strftime('%Y-%m-%d %H:%M:%S')
    })

df = pd.DataFrame(favorites)
df.to_csv("favorites.csv", index=False, encoding="utf-8")

print("Done! Data saved to favorites.csv")
```

Figure 2: The favorites.py script

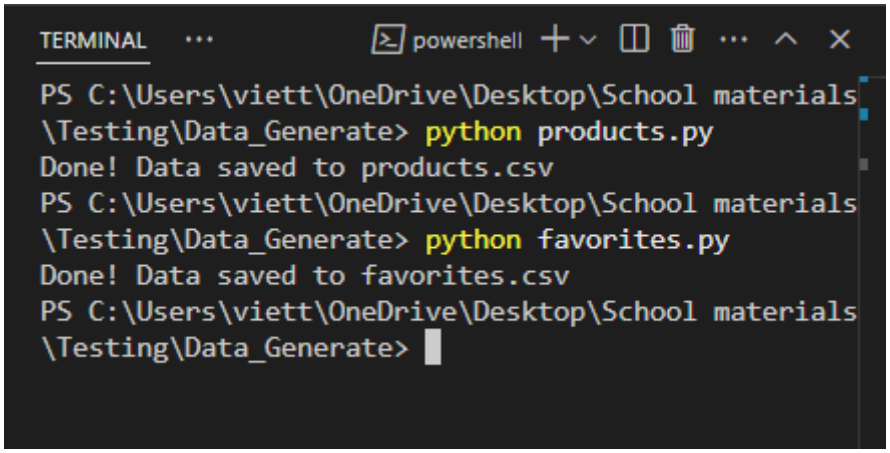

Figure 3: Running the products.py and favorites.py